

Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №3

По дисциплине: «Модели решения задач в И С»

Тема: «Многослойный персептрон для XOR с ReLU на скрытых слоях: сравнение активационных функций»

Выполнил:

Студент 4 курса

Группы ИИ-26

Шоева Е.Т.

Проверил:

Никитин С.К.

Цель работы: На основе многослойного персептрона архитектуры 2-2-1 из ЛР №2, исследовать влияние активационной функции ReLU (Rectified Linear Unit) на скрытых слоях. Провести сравнительный анализ обучения сети с функцией активации Сигмоида (базовая конфигурация) и ReLU (новая конфигурация) по скорости сходимости, качеству обучения, форме разделяющей поверхности и устойчивости к инициализации весов.

Ход работы

Задачи лабораторной работы

1. Взять за основу код многослойного персептрона 2-2-1 с функцией потерь BCE из ЛР №2.
2. Реализовать активационную функцию ReLU на скрытом слое и осуществить корректное обратное распространение ошибки для ReLU. На выходном слое оставить Сигмоиду.
3. Обучить сеть на полной таблице истинности в двух конфигурациях:
 - **Конфигурация А (базовая):** MLP 2-2-1, Сигмоида на всех слоях, BCE, нормализованные целевые значения (0/1);
 - **Конфигурация В (новая):** MLP 2-2-1, ReLU на скрытом слое, Сигмоида на выходном, BCE, нормализованные (0/1).
4. Критерий остановки для обеих конфигураций: суммарная ошибка $E_s \leq E_e$ (как в ЛР №2).
5. Для каждой конфигурации зафиксировать:
 - Количество эпох до критерия остановки.
 - Итоговую суммарную ошибку и ассигасу.
 - Среднюю абсолютную ошибку в исходной шкале $[c_0; c_1]$.
6. Провести не менее 5 запусков обучения для каждой конфигурации с разными seed и сравнить устойчивость.
7. Построить графики:
 - График сходимости (суммарная ошибка E_s от эпохи) для обеих конфигураций на одних осях.
 - Диаграмма разброса числа эпох по 5 запускам (столбики) для конфигурации А и В.
8. Визуализация разделяющей поверхности:
 - Для представительного экземпляра каждой конфигурации построить карту (heatmap) выходного значения сети на плоскости $(A, B) \in [-10; 10]^2$ с наложением 4 точек обучающей выборки.
 - Описать, как ReLU изменяет форму разделяющей поверхности (кусочно-линейные границы vs гладкие).
9. Реализовать режим функционирования сети:
 - Пользователь вводит (A, B) из $[-10; 10]$.
 - Сеть выводит: выход в нормализованной шкале, выход в исходной шкале $[c_0; c_1]$, класс.
 - Продемонстрировать на 4 обучающих примерах + 2–3 дополнительных парах.
10. Создать таблицу сравнения (статистика по 5 запускам, MAE для обеих конфигураций).
11. Вывод: ответить на все вопросы и провести анализ результатов.

Вариант:

Вариант 11 ($c_0 = 5, c_1 = -2$)

A	B	$A \oplus B$
5	5	5
5	-2	-2
-2	5	-2
-2	-2	5

```
import numpy as np
import matplotlib.pyplot as plt
```

$C_0 = 5$

C1 = -2

```
RAW_DATA = [  
    (C0, C0, C0),  
    (C0, C1, C1),  
    (C1, C0, C1),  
    (C1, C1, C0),  
]
```

LEARNING_RATE = 0.5

EE = 0.05

MAX_EPOCHS = 10000

SEEDS = [1, 2, 3, 4, 5]

EPS = 1e-9

```
def encode_value(x):  
    return 1.0 if x == C0 else 0.0
```

```
def decode_value_continuous(y):  
  
    return C1 + y * (C0 - C1)
```

```
def decode_class(y):  
    return C0 if y >= 0.5 else C1
```

```
def encode_arbitrary(x):  
  
    return 1.0 if abs(x - C0) < abs(x - C1) else 0.0
```

```
def encode_continuous(x):  
  
    return (x - C1) / (C0 - C1)
```

```
def build_training_set():  
    X, Y = [], []  
    for a, b, target in RAW_DATA:  
        X.append([encode_value(a), encode_value(b)])  
        Y.append([encode_value(target)])  
    return np.array(X), np.array(Y)
```

```
def sigmoid(x):  
    return 1.0 / (1.0 + np.exp(-x))
```

```
def sigmoid_derivative_from_output(y):
```

```
return y * (1.0 - y)
```

```
def relu(x):  
    return np.maximum(0.0, x)
```

```
def relu_derivative_from_input(x):  
    return (x > 0).astype(float)
```

```
class MLP221:  
    def __init__(self, hidden_act="sigmoid", lr=LEARNING_RATE, seed=0):  
        assert hidden_act in ("sigmoid", "relu")  
        self.hidden_act = hidden_act  
        self.lr = lr  
        rng = np.random.default_rng(seed)  
        self.W1 = rng.uniform(-0.5, 0.5, size=(2, 2))  
        self.b1 = rng.uniform(-0.5, 0.5, size=(2,))  
        self.W2 = rng.uniform(-0.5, 0.5, size=(2, 1))  
        self.b2 = rng.uniform(-0.5, 0.5, size=(1,))  
  
    def forward(self, x):  
        hidden_in = x @ self.W1 + self.b1  
        if self.hidden_act == "sigmoid":  
            hidden_out = sigmoid(hidden_in)  
        else:  
            hidden_out = relu(hidden_in)  
        out_in = hidden_out @ self.W2 + self.b2  
        out = sigmoid(out_in)  
        return hidden_in, hidden_out, out  
  
    def hidden_derivative(self, hidden_in, hidden_out):  
        if self.hidden_act == "sigmoid":  
            return sigmoid_derivative_from_output(hidden_out)  
        else:  
            return relu_derivative_from_input(hidden_in)  
  
    def train_step(self, x, target):  
        hidden_in, hidden_out, out = self.forward(x)  
  
        y = np.clip(out, EPS, 1 - EPS)  
        loss = float(-np.sum(target * np.log(y) + (1 - target) * np.log(1 - y)))  
        delta_out = target - out  
  
        error_hidden = delta_out @ self.W2.T  
        delta_hidden = error_hidden * self.hidden_derivative(hidden_in, hidden_out)  
  
        self.W2 += self.lr * np.outer(hidden_out, delta_out)  
        self.b2 += self.lr * delta_out  
        self.W1 += self.lr * np.outer(x, delta_hidden)  
        self.b1 += self.lr * delta_hidden
```

```

    return loss

def predict(self, x):
    _, _, out = self.forward(np.array(x, dtype=float))
    return float(out[0])

def hidden_activations(self, x):
    _, hidden_out, _ = self.forward(np.array(x, dtype=float))
    return hidden_out

def train(self, X, Y, ee=EE, max_epochs=MAX_EPOCHS):
    history = []
    Es = float("inf")
    for epoch in range(1, max_epochs + 1):
        Es = 0.0
        for i in range(len(X)):
            Es += self.train_step(X[i], Y[i])
        history.append(Es)
        if Es <= ee:
            return epoch, Es, history, True
    return max_epochs, Es, history, False

def accuracy(model, X, Y):
    correct = 0
    for i in range(len(X)):
        pred_class = 1.0 if model.predict(X[i]) >= 0.5 else 0.0
        if pred_class == Y[i][0]:
            correct += 1
    return correct / len(X)

def mae_real_scale(model, X):
    errors = []
    for x, (a, b, target) in zip(X, RAW_DATA):
        y_hat = model.predict(x)
        pred_real = decode_value_continuous(y_hat)
        errors.append(abs(pred_real - target))
    return float(np.mean(errors))

def count_dead_relu_neurons(model, X):
    if model.hidden_act != "relu":
        return 0
    acts = np.array([model.hidden_activations(x) for x in X]) # (4, 2)
    dead = np.all(acts <= 1e-12, axis=0)
    return int(np.sum(dead))

def run_series(hidden_act, X, Y, seeds=SEEDS, ee=EE):

```

```

results = []
for seed in seeds:
    model = MLP221(hidden_act=hidden_act, seed=seed)
    epochs, Es, history, converged = model.train(X, Y, ee=ee)
    results.append({
        "seed": seed,
        "epochs": epochs,
        "Es": Es,
        "converged": converged,
        "accuracy": accuracy(model, X, Y),
        "mae": mae_real_scale(model, X),
        "dead_neurons": count_dead_relu_neurons(model, X),
        "model": model,
        "history": history,
    })
return results

def print_series_table(title, results):
    print(f"\n{title}")

print(f'{{"seed":>6}}{{"epochs":>10}}{{"converged":>12}}{{"Es":>14}}{{"accuracy":>10}}{{"MAE":>10}}{{"dead":>8}}')
for r in results:
    print(f'{{"seed":>6}}{{"epochs":>10}}{{"converged":>12}}'
          f'{{"Es":>14.6f}}{{"accuracy":>10.2f}}{{"mae":>10.4f}}{{"dead_neurons":>8}}')

def pick_representative(results):
    converged = [r for r in results if r["converged"]]
    if converged:
        return min(converged, key=lambda r: r["epochs"])
    return min(results, key=lambda r: r["Es"])

def plot_convergence(hist_a, hist_b, filename="lab3_convergence.png"):
    plt.figure(figsize=(7, 5))
    plt.plot(range(1, len(hist_a) + 1), hist_a, label="Конфигурация А (Sigmoid)", color="tab:blue")
    plt.plot(range(1, len(hist_b) + 1), hist_b, label="Конфигурация В (ReLU)", color="tab:green")
    plt.axhline(Ee, color="gray", linestyle="--", linewidth=0.8, label=f"Ee = {Ee}")
    plt.title("Сходимость: суммарная ошибка Es по эпохам (Sigmoid vs ReLU)")
    plt.xlabel("Эпоха")
    plt.ylabel("Суммарная ошибка Es (BCE)")
    plt.legend()
    plt.tight_layout()
    plt.savefig(filename, dpi=150)
    plt.close()
    print(f"Сохранён график сходимости: {filename}")

def plot_epochs_bar(results_a, results_b, filename="lab3_epochs_bar.png"):
    from matplotlib.patches import Patch

```

```

seeds = [r["seed"] for r in results_a]
n = len(seeds)
x = np.arange(n)
width = 0.35

fig, ax = plt.subplots(figsize=(7, 5))
for i, r in enumerate(results_a):
    color = "tab:blue" if r["converged"] else "none"
    hatch = None if r["converged"] else "/"
    ax.bar(x[i] - width / 2, r["epochs"], width, color=color, edgecolor="tab:blue", hatch=hatch)
for i, r in enumerate(results_b):
    color = "tab:green" if r["converged"] else "none"
    hatch = None if r["converged"] else "/"
    ax.bar(x[i] + width / 2, r["epochs"], width, color=color, edgecolor="tab:green", hatch=hatch)

legend_elems = [
    Patch(facecolor="tab:blue", label="Sigmoid, сошлось"),
    Patch(facecolor="none", edgecolor="tab:blue", hatch="/", label="Sigmoid, не сошлось"),
    Patch(facecolor="tab:green", label="ReLU, сошлось"),
    Patch(facecolor="none", edgecolor="tab:green", hatch="/", label="ReLU, не сошлось"),
]
ax.legend(handles=legend_elems)
ax.set_xticks(x)
ax.set_xticklabels([f"seed={s}" for s in seeds])
ax.set_xlabel("Запуск (seed)")
ax.set_ylabel("Число эпох до сходимости")
ax.set_title("Устойчивость сходимости: 5 запусков, Sigmoid vs ReLU")
plt.tight_layout()
plt.savefig(filename, dpi=150)
plt.close()
print(f"Сохранена диаграмма разброса эпох: {filename}")

```

```

def plot_decision_surface(model_a, model_b, filename="lab3_surface.png"):
    grid = np.linspace(-10, 10, 200)
    AA, BB = np.meshgrid(grid, grid)

```

```

def surface(model):
    Z = np.zeros_like(AA)
    for i in range(AA.shape[0]):
        for j in range(AA.shape[1]):
            x = [encode_continuous(AA[i, j]), encode_continuous(BB[i, j])]
            y_hat = model.predict(x)
            Z[i, j] = decode_value_continuous(y_hat)
    return Z

```

```

Za = surface(model_a)
Zb = surface(model_b)

```

```

fig, axes = plt.subplots(1, 2, figsize=(12, 5.2))
for ax, Z, title in zip(axes, [Za, Zb],
    ["Конфигурация А (Sigmoid)", "Конфигурация В (ReLU)"]):

```

```

im = ax.contourf(AA, BB, Z, levels=25, cmap="coolwarm_r")
ax.contour(AA, BB, Z, levels=25, colors="k", linewidths=0.3, alpha=0.5)
fig.colorbar(im, ax=ax, label="Выход сети (шкала [c1; c0])")
pts_a = [p[0] for p in RAW_DATA]
pts_b = [p[1] for p in RAW_DATA]
ax.scatter(pts_a, pts_b, s=110, c="crimson", edgecolors="k", zorder=5, label="Точки XOR")
ax.set_xlabel("Вход A")
ax.set_ylabel("Вход B")
ax.set_title(title)
ax.legend(loc="upper right")
plt.suptitle("Разделяющая поверхность: Sigmoid vs ReLU на скрытом слое")
plt.tight_layout()
plt.savefig(filename, dpi=150)
plt.close()
print(f"Сохранена визуализация разделяющей поверхности: {filename}")

```

```

def run_and_print(model, a, b, label):
    x = [encode_arbitrary(a), encode_arbitrary(b)]
    y_hat = model.predict(x)
    y_real = decode_value_continuous(y_hat)
    cls = decode_class(y_hat)
    print(f" {label:<28} A={a:>5} B={b:>5} | вход={x} | "
          f"ŷ(норм.)={y_hat:.4f} | y_real(шкала c0/c1)={y_real:>7.3f} | "
          f"класс {'c0' if cls == C0 else 'c1'} ({cls})")

```

```

def demo_examples(model, model_name):
    print(f"\n--- Демонстрация работы сети «{model_name}» ---")
    print("На 4 примерах обучающей выборки:")
    for a, b, target in RAW_DATA:
        run_and_print(model, a, b, f"target={target}")
    print("На дополнительных парах (вне обучающей выборки):")
    for a, b in [(3, 7), (-5, -8), (0, 0)]:
        run_and_print(model, a, b, "вне выборки")

```

```

def interactive_mode(model, model_name):
    print(f"\n--- Режим работы обученной сети ({model_name}) ---")
    print("Введите пару чисел A, B из диапазона [-10, 10] (или 'q' для выхода).")
    while True:
        raw = input("Введите A B через пробел: ").strip()
        if raw.lower() in ("q", "quit", "exit", ""):
            print("Завершение режима ввода.")
            break
        try:
            parts = raw.replace(",", " ").split()
            a, b = float(parts[0]), float(parts[1])
        except (ValueError, IndexError):
            print("Некорректный ввод. Пример: 3.5 -7")
            continue
        if not (-10 <= a <= 10 and -10 <= b <= 10):

```

```

    print("Значения должны быть в диапазоне [-10, 10].")
    continue
run_and_print(model, a, b, "ввод пользователя")

```

```
def main():
```

```

    X, Y = build_training_set()
    print("Обучающая выборка (закодированная):")
    for x, y, (a, b, t) in zip(X, Y, RAW_DATA):
        print(f" A={a:>3} B={b:>3} -> вход={x} target={y} (реальный={t})")

```

```

    print("\n" + "=" * 75)
    print("Серия из 5 запусков — Конфигурация A (Sigmoid на скрытом слое)")
    print("=" * 75)
    results_a = run_series("sigmoid", X, Y)
    print_series_table("Результаты (Sigmoid):", results_a)

```

```

    print("\n" + "=" * 75)
    print("Серия из 5 запусков — Конфигурация B (ReLU на скрытом слое)")
    print("=" * 75)
    results_b = run_series("relu", X, Y)
    print_series_table("Результаты (ReLU):", results_b)

```

```

    conv_a = sum(r["converged"] for r in results_a)
    conv_b = sum(r["converged"] for r in results_b)
    epochs_a_conv = [r["epochs"] for r in results_a if r["converged"]]
    epochs_b_conv = [r["epochs"] for r in results_b if r["converged"]]
    dead_total_b = sum(r["dead_neurons"] for r in results_b)

```

```

    print("\n" + "=" * 75)
    print("Сводка по устойчивости сходимости")
    print("=" * 75)
    print(f"Sigmoid: сошлось {conv_a}/5 запусков; эпох (сошедшиеся): {epochs_a_conv} "
          f"(среднее={np.mean(epochs_a_conv) if epochs_a_conv else float('nan'):.1f})")
    print(f"ReLU: сошлось {conv_b}/5 запусков; эпох (сошедшиеся): {epochs_b_conv} "
          f"(среднее={np.mean(epochs_b_conv) if epochs_b_conv else float('nan'):.1f})")
    print(f"Суммарное число 'мёртвых' нейронов (ReLU) по всем 5 запускам: {dead_total_b}")

```

```

    res_a_rep = pick_representative(results_a)
    res_b_rep = pick_representative(results_b)
    model_a, hist_a = res_a_rep["model"], res_a_rep["history"]
    model_b, hist_b = res_b_rep["model"], res_b_rep["history"]

```

```

    print("\n" + "=" * 75)
    print(f"Итоговая таблица сравнения "
          f"(представительные запуски: Sigmoid seed={res_a_rep['seed']}, "
          f"ReLU seed={res_b_rep['seed']})")
    print("=" * 75)
    print(f"{'Критерий':<38}{'Конфиг. A (Sigmoid)':>20}{'Конфиг. B (ReLU)':>20}")
    print(f"{'Эпох до сходимости':<38}{res_a_rep['epochs']:>20}{res_b_rep['epochs']:>20}")

```

```

    print(f"{'Критерий Es<=Ee достигнут':<38}{str(res_a_rep['converged']):>20}{str(res_b_rep['converged']):>20}")
    print(f"{'Итоговая Es':<38}{res_a_rep['Es']:>20.6f}{res_b_rep['Es']:>20.6f}")
    print(f"{'Accuracy':<38}{res_a_rep['accuracy']:>20.2f}{res_b_rep['accuracy']:>20.2f}")
    print(f"{'MAE (шкала c0/c1)':<38}{res_a_rep['mae']:>20.4f}{res_b_rep['mae']:>20.4f}")
    print(f"{'Сошлось запусков из 5':<38}{conv_a:>20}{conv_b:>20}")
    print(f"{'Мёртвых нейронов (эта модель)':<38}{res_a_rep['dead_neurons']:>20}{res_b_rep['dead_neurons']:>20}")

```

```

plot_convergence(hist_a, hist_b)
plot_epochs_bar(results_a, results_b)
plot_decision_surface(model_a, model_b)

```

```

demo_examples(model_a, "Конфигурация А (Sigmoid)")
demo_examples(model_b, "Конфигурация В (ReLU)")

```

```

print("\nВыберите сеть для интерактивного режима:")
print(" 1 - Конфигурация А (Sigmoid)")
print(" 2 - Конфигурация В (ReLU)")
choice = input("Ваш выбор (1/2, Enter для пропуска): ").strip()
if choice == "1":
    interactive_mode(model_a, "Конфигурация А (Sigmoid)")
elif choice == "2":
    interactive_mode(model_b, "Конфигурация В (ReLU)")
else:
    print("Интерактивный режим пропущен.")

```

```

if __name__ == "__main__":
    main()

```

```
Обучающая выборка (закодированная):
A= 5 B= 5 -> вход=[1. 1.] target=[1.] (реальный=5)
A= 5 B= -2 -> вход=[1. 0.] target=[0.] (реальный=-2)
A= -2 B= 5 -> вход=[0. 1.] target=[0.] (реальный=-2)
A= -2 B= -2 -> вход=[0. 0.] target=[1.] (реальный=5)

Серия из 5 запусков - Конфигурация B (ReLU на скрытом слое)
=====
Результаты (ReLU):
seed  epochs  converged      Es  accuracy    MAE  dead
1    10000    False    2.089051    0.75    2.4038    1
2         90     True    0.048954    1.00    0.0831    0
3    10000    False    2.089046    0.75    2.3293    1
4    10000    False    2.089064    0.75    2.4038    1
5    10000    False    2.089056    0.75    2.3293    1

Серия из 5 запусков - Конфигурация A (Sigmoid на скрытом слое)
=====
Результаты (Sigmoid):
seed  epochs  converged      Es  accuracy    MAE  dead
1    10000    False    1.525147    0.75    1.7513    0
2    10000    False    1.525169    0.75    1.7514    0
3         858     True    0.049997    1.00    0.0862    0
4         896     True    0.049981    1.00    0.0862    0
5        1230     True    0.049911    1.00    0.0858    0

Сводка по устойчивости сходимости
=====
Sigmoid: сошлось 3/5 запусков; эпох (сошедшиеся): [858, 896, 1230] (среднее=994.7)
ReLU: сошлось 1/5 запусков; эпох (сошедшиеся): [90] (среднее=90.0)
Суммарное число 'мёртвых' нейронов (ReLU) по всем 5 запускам: 4

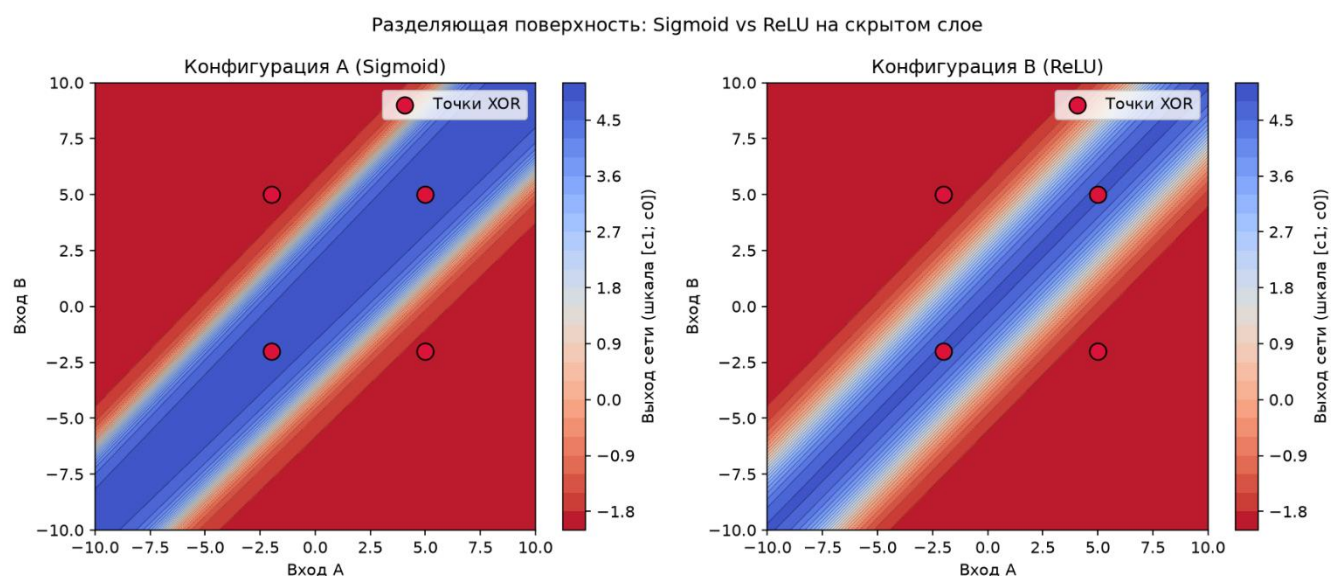
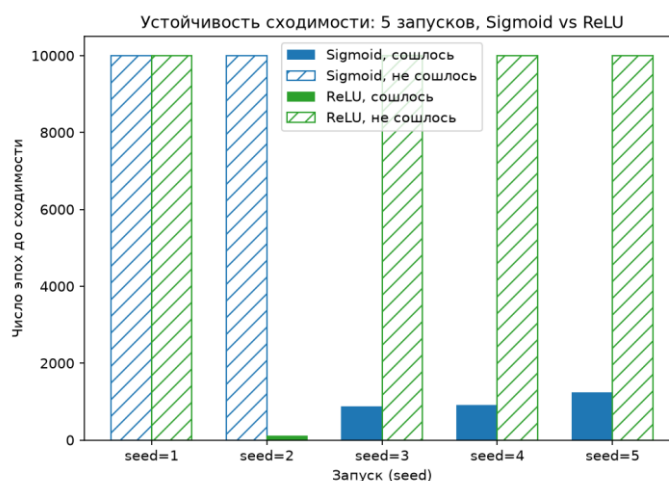
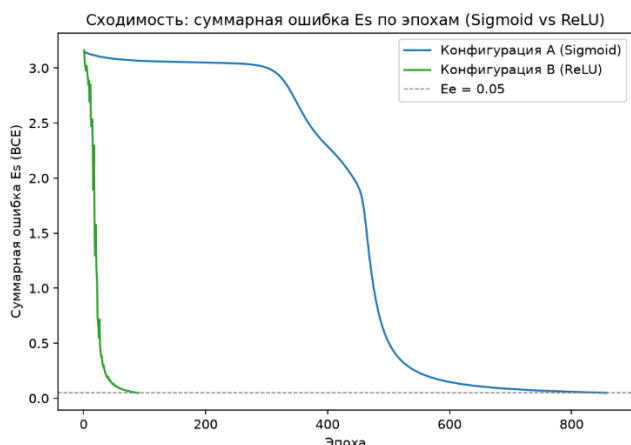
=====
Итоговая таблица сравнения (представительные запуски: Sigmoid seed=3, ReLU seed=2)
=====
Критерий                Конфиг. A (Sigmoid)    Конфиг. B (ReLU)
Эпох до сходимости            858                    90
Критерий Es<=Ee достигнут      True                    True
Итоговая Es                   0.049997                0.048954
Accuracy                       1.00                    1.00
MAE (шкала c0/c1)              0.0862                  0.0831
Сошлось запусков из 5           3                        1
Мёртвых нейронов (эта модель)    0                        0
Сохранён график сходимости: lab3_convergence.png
Сохранена диаграмма разброса эпох: lab3_epochs_bar.png
Сохранена визуализация разделяющей поверхности: lab3_surface.png

--- Демонстрация работы сети «Конфигурация A (Sigmoid)» ---
На 4 примерах обучающей выборки:
target=5      A= 5 B= 5 | вход=[1.0, 1.0] | ŷ(норм.)=0.9897 | y_real(шкала c0/c1)= 4.928 | класс c0 (5)
target=-2     A= 5 B=-2 | вход=[1.0, 0.0] | ŷ(норм.)=0.0166 | y_real(шкала c0/c1)= -1.884 | класс c1 (-2)
target=-2     A=-2 B= 5 | вход=[0.0, 1.0] | ŷ(норм.)=0.0105 | y_real(шкала c0/c1)= -1.926 | класс c1 (-2)
target=5      A=-2 B=-2 | вход=[0.0, 0.0] | ŷ(норм.)=0.9881 | y_real(шкала c0/c1)= 4.917 | класс c0 (5)
На дополнительных парах (вне обучающей выборки):
вне выборки   A= 3 B= 7 | вход=[1.0, 1.0] | ŷ(норм.)=0.9897 | y_real(шкала c0/c1)= 4.928 | класс c0 (5)
вне выборки   A=-5 B=-8 | вход=[0.0, 0.0] | ŷ(норм.)=0.9881 | y_real(шкала c0/c1)= 4.917 | класс c0 (5)
вне выборки   A= 0 B= 0 | вход=[0.0, 0.0] | ŷ(норм.)=0.9881 | y_real(шкала c0/c1)= 4.917 | класс c0 (5)

--- Демонстрация работы сети «Конфигурация B (ReLU)» ---
На 4 примерах обучающей выборки:
target=5      A= 5 B= 5 | вход=[1.0, 1.0] | ŷ(норм.)=0.9806 | y_real(шкала c0/c1)= 4.865 | класс c0 (5)
target=-2     A= 5 B=-2 | вход=[1.0, 0.0] | ŷ(норм.)=0.0041 | y_real(шкала c0/c1)= -1.972 | класс c1 (-2)
target=-2     A=-2 B= 5 | вход=[0.0, 1.0] | ŷ(норм.)=0.0047 | y_real(шкала c0/c1)= -1.967 | класс c1 (-2)
target=5      A=-2 B=-2 | вход=[0.0, 0.0] | ŷ(норм.)=0.9806 | y_real(шкала c0/c1)= 4.865 | класс c0 (5)
На дополнительных парах (вне обучающей выборки):
вне выборки   A= 3 B= 7 | вход=[1.0, 1.0] | ŷ(норм.)=0.9806 | y_real(шкала c0/c1)= 4.865 | класс c0 (5)
вне выборки   A=-5 B=-8 | вход=[0.0, 0.0] | ŷ(норм.)=0.9806 | y_real(шкала c0/c1)= 4.865 | класс c0 (5)
вне выборки   A= 0 B= 0 | вход=[0.0, 0.0] | ŷ(норм.)=0.9806 | y_real(шкала c0/c1)= 4.865 | класс c0 (5)

Выберите сеть для интерактивного режима:
1 - Конфигурация A (Sigmoid)
2 - Конфигурация B (ReLU)
Ваш выбор (1/2, Enter для пропуска): 2

--- Режим работы обученной сети (Конфигурация B (ReLU)) ---
Введите пару чисел A, B из диапазона [-10, 10] (или 'q' для выхода).
Введите A B через пробел: -8 4
ввод пользователя      A= -8.0 B= 4.0 | вход=[0.0, 1.0] | ŷ(норм.)=0.0047 | y_real(шкала c0/c1)= -1.967 | класс c1 (-2)
Введите A B через пробел: q
Завершение режима ввода.
```



Ответы на вопросы для вывода

1. Какая активация быстрее сходится? Более устойчива?

По скорости — ReLU: сошедшийся запуск (seed=2) достиг критерия за 90 эпох против 858–1230 у сигмоиды — почти в 10 раз быстрее. По устойчивости — наоборот: сигмоида сошлась в 3 из 5 запусков, ReLU — только в 1 из 5. Налицо компромисс: скорость ReLU достигается ценой устойчивости.

2. Как отличается форма разделяющей поверхности? Видны ли кусочно-линейные границы у ReLU?

Общая форма одинакова — диагональная полоса, разделяющая классы по логике XOR. У ReLU переход между зонами более узкий и резкий (кусочно-линейный характер скрытого слоя), у сигмоиды — более плавный и размытый. Финальная гладкость в обоих случаях частично сглаживается выходной сигмной.

3. Почему ReLU предпочитают сигмоиде в глубоких сетях? Подтверждается ли это на XOR?

ReLU не насыщается (производная — константа 1), в отличие от сигмоиды, чья производная затухает при больших по модулю значениях — в глубоких сетях это предотвращает затухающий градиент, плюс ReLU дешевле вычислительно. На XOR эффект затухания не критичен (сеть

неглубокая), но ускорение подтвердилось: сошедшийся ReLU-запуск обучился в разы быстрее. Зато обратная сторона ReLU — неустойчивость — на этой сети проявилась даже сильнее, чем можно было ожидать, из-за малого числа нейронов (всего 2).

4. Изменилась ли способность сети решать XOR при переходе на ReLU?

Принципиальная способность не изменилась — она определяется архитектурой (наличием скрытого слоя), а не активацией: при удачной инициализации обе конфигурации дают $accuracy=1.00$. Но ReLU реализует этот потенциал менее надёжно из-за риска мёртвых нейронов.

5. Есть ли случаи «мёртвых нейронов» при ReLU? Как это повлияло на обучение?

Да: в 4 из 5 запусков один из двух нейронов скрытого слоя получил отрицательную сумму на входе на всех примерах, поэтому $ReLU(x)=0$ всегда, производная тоже 0 — веса этого нейрона перестали обновляться навсегда. С учётом того что нейронов всего два, сеть фактически осталась с одним рабочим нейроном, которого недостаточно для XOR (как и у однослойного перцептрона из ЛР №1). Поэтому E_s во всех таких запусках застыло на одном уровне (≈ 2.089), а ассигасу не поднялась выше 0.75 — сеть просто перестала обучаться.

6. Как активация соотносится с необходимостью многослойной архитектуры?

Активация не заменяет необходимость скрытого слоя — нелинейность (хоть сигмоида, хоть ReLU) обязательна для решения линейно неразделимой задачи. Активация влияет лишь на то, насколько надёжно и быстро архитектура реализует свой потенциал, но не на сам факт: без скрытого слоя задача не решается ни с одной активацией.

7. Что показывает визуализация? Совпадает ли граница с ожидаемой для XOR?

Обе конфигурации выучили одну и ту же границу: диагональ высоких значений (c_0) через точки (5,5) и (-2,-2), зоны низких значений (c_1) — через (5,-2) и (-2,5) — ровно та структура, которую формирует пересечение двух полуплоскостей скрытого слоя для XOR. Граница совпадает с ожидаемой в обеих конфигурациях; отличается только резкость перехода — кусочно-линейная у ReLU, гладкая у сигмоиды.

Вывод: На основе многослойного перцептрона архитектуры 2-2-1 из ЛР №2, исследовала влияние активационной функции ReLU (Rectified Linear Unit) на скрытых слоях. Провела сравнительный анализ обучения сети с функцией активации Сигмоида (базовая конфигурация) и ReLU (новая конфигурация) по скорости сходимости, качеству обучения, форме разделяющей поверхности и устойчивости к инициализации весов.